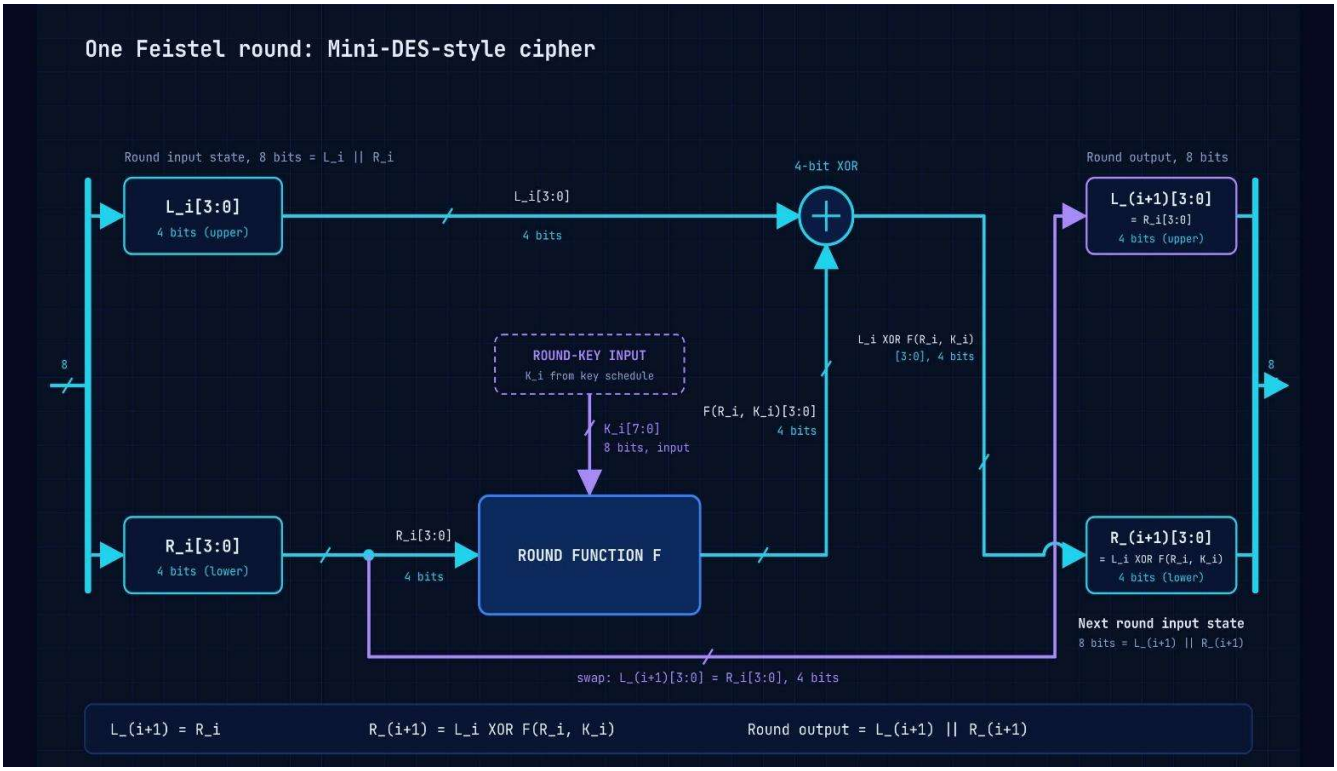
DLD Project

DES Style Feistel Network Cipher

STAGE -3

1.Final Design

Feistel round:



A Feistel round divides the 16-bit input into two 8-bit halves, Left (*L*) and Right (*R*). The right half undergoes key mixing, S-box substitution, and P-box permutation. Its output is XORed with the left half to generate the new right half, while the old right half becomes the new left half. Three rounds are performed using different round keys to produce the final 16-bit ciphertext.

Complete design:

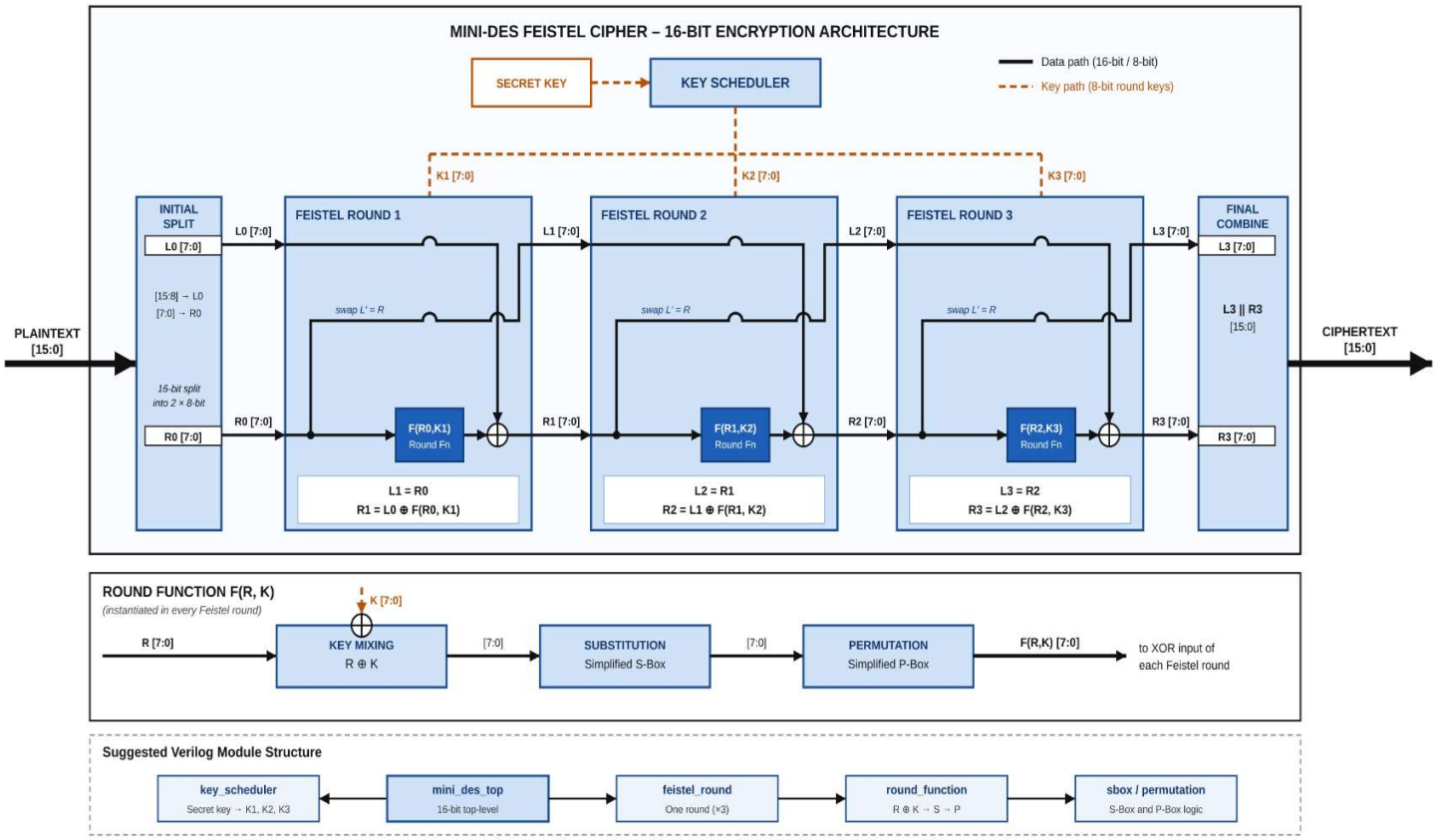
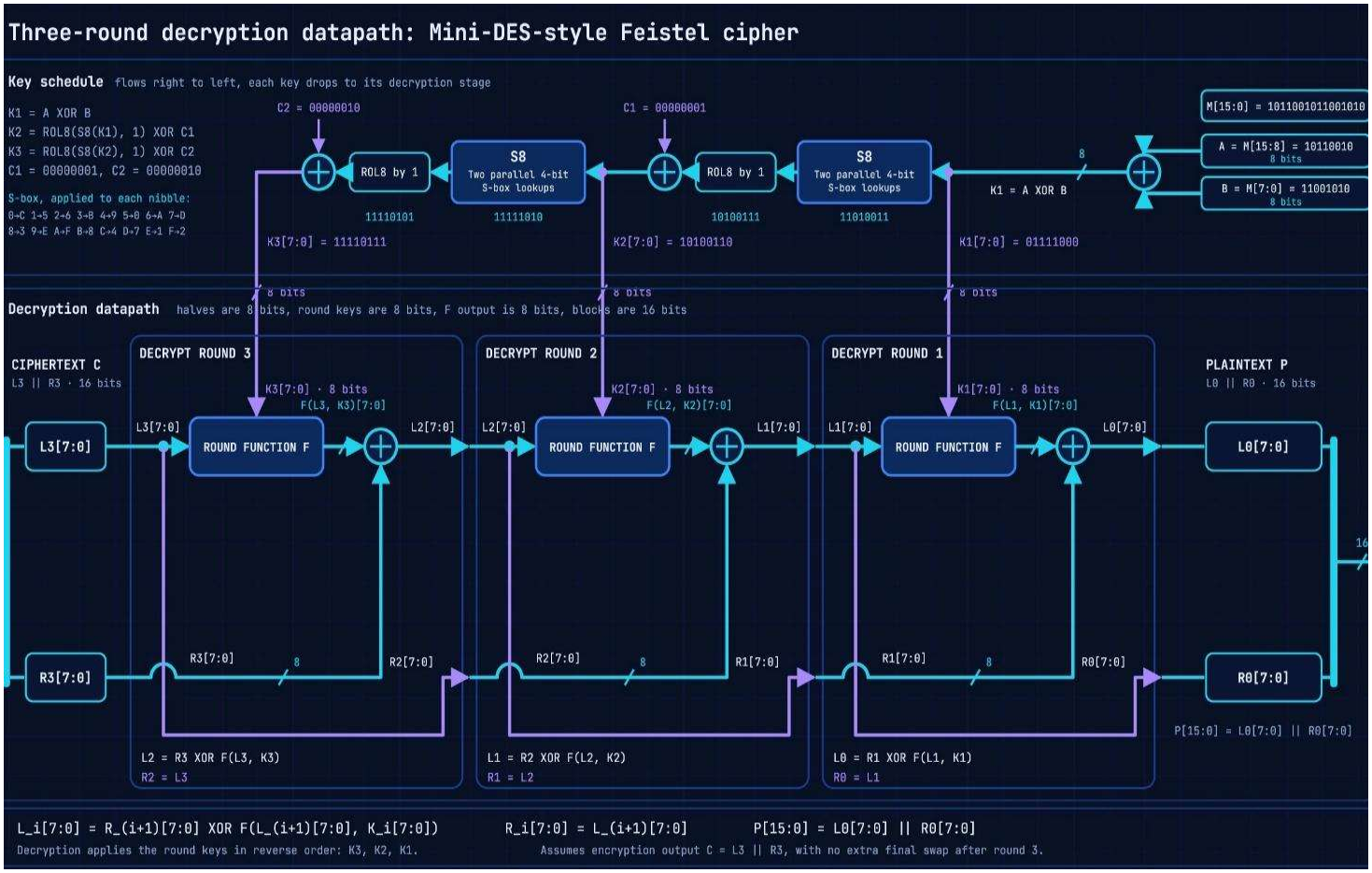


Figure 1. Preliminary architecture of the 16-bit Mini-DES Feistel cipher (3 rounds, 8-bit halves, 8-bit round keys).

Decrypter:



The decryptor reverses the encryption process to recover the original 16-bit plaintext from the ciphertext. It applies the same Feistel round structure but uses the round keys in reverse order, \(K_3\), \(K_2\), and \(K_1\). Since Feistel networks are reversible, the original plaintext can be recovered by applying the inverse rounds, without requiring the S-box itself to be inverted.

2.Complete Implementation

We have implemented **Feistel Round**, **Round Function** and **Key scheduler** part of project. In Round function, **S box and P box** were implemented. All these were designed and tested separately in Verilog testbench and finally incorporated into one design that is **Mini-Des**.

Attached below is git-hub link:

https://github.com/2025csb1156-yoyo/Feistel-Network-Cipher-Mini-DES-Style-

Attached below are links of EDA playground

Mini-Des

Decrypter

feistel function

key scheduling

Round function

S box

P BOX:

3. Complete Design Logic

The Mini-Des encryption takes a 16-bit input text, split it into right and left of 8-bit each and these two will be input to first Feistel round which will also have one 8-bit key1 (received from key scheduler using a 16-bit master key). Now right part and key will be sent to Round function (which do key mixing, s box and p box) and 8-bit output of round function will be finally XOR with left part and this will be "right output" and initial right part will be "left output".

Now right and left output of first Feistel round would be sent to next Feistel round along with the key2 and its output will be further sent to third Feistel round with key3 and finally right and left part will be joined to get the encrypted text.

4. Simulation Results

Mini DES:

```
          Master Key = A3F7
| Input | Expected | Actual | Status |
-----------------------------------------------
| 1234 | 6646 | 6646 | PASS |
| 0000 | 885f | 885f | PASS |
| ffff | 5c88 | 5c88 | PASS |
| a5c3 | 8d69 | 8d69 | PASS |
| beef | 1bc2 | 1bc2 | PASS |
| a3f7 | 8076 | 8076 | PASS |
| 0001 | 13da | 13da | PASS |
| 0100 | da40 | da40 | PASS |
| 55aa | 3b06 | 3b06 | PASS |
| dead | 74d8 | 74d8 | PASS |
| 8080 | 2f66 | 2f66 | PASS |
| 2468 | 1f91 | 1f91 | PASS |
-----------------------------------------------
Total tests : 12
Passed      : 12
Failed      : 0
```

Feistel round:

```
        Feistel Round VERIFICATION
R1  L1      K1      R2||L2      Expected    Result
-----------------------------------------
00  00      00      6f00        6f00        Pass
ff  ff      ff      90ff        90ff        Pass
aa  55      aa      3aaa        3aaa        Pass
55  aa      55      c555        c555        Pass
12  34      56      7012        7012        Pass
ab  cd      ef      89ab        89ab        Pass
00  a5      5a      6a00        6a00        Pass
3c  c3      00      513c        513c        Pass
-----------------------------------------
All tests PASSED.
testbench.sv:66: $finish called at 80000 (1ps)
```

Decrypter :

```
            Mini-DES style Feistel Decryptor
                    Master Key = A3F7
| Ciphertext | Expected Plaintext | Actual Output | Status |
------------------------------------------------------------
| 6646  |  f5d6   |  f5d6  |  PASS  |
| 885f  |  3614   |  3614  |  PASS  |
| 5c88  |  3002   |  3002  |  PASS  |
| 8d69  |  90f4   |  90f4  |  PASS  |
| 1bc2  |  a8f4   |  a8f4  |  PASS  |
| 8076  |  9dd2   |  9dd2  |  PASS  |
| 13da  |  4cce   |  4cce  |  PASS  |
| da40  |  5f30   |  5f30  |  PASS  |
| 3b06  |  1714   |  1714  |  PASS  |
| 74d8  |  3d06   |  3d06  |  PASS  |
| 2f66  |  9264   |  9264  |  PASS  |
| 1f91  |  340a   |  340a  |  PASS  |
------------------------------------------------------------
Total tests : 12
Passed      : 12
Failed      : 0
```

As original plaintext is first encrypted then decrypted back to plaintext

the project is working as intended

5. Performance analysis

The Mini-DES design incorporates multiple layers of security through its three-round Feistel structure. Each round uses a different round key generated by a nonlinear key schedule, making the encryption process dependent on both the plaintext and the master key. XOR-based key mixing introduces key dependency, S-box substitution provides nonlinearity, and P-box permutation rearranges bits to help spread the influence of input changes across the block. Repeating these operations over three rounds increases the complexity of the transformation.

6. Work done by team:

IZYAN ZUBERI- Report making, overall management.

LAKSHAY K PADAM- Block diagrams, mini-des.

AKANKSHA KUMARI- Feistel function, descryptor.

HASRSHVARDHAN- Feistel function, mini-des.

ASHLYN ALDRIN JOSE- Verification and debugging.

